Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Área de Informática

Estructura de Datos II

Catálogo de Libros

Manual técnico

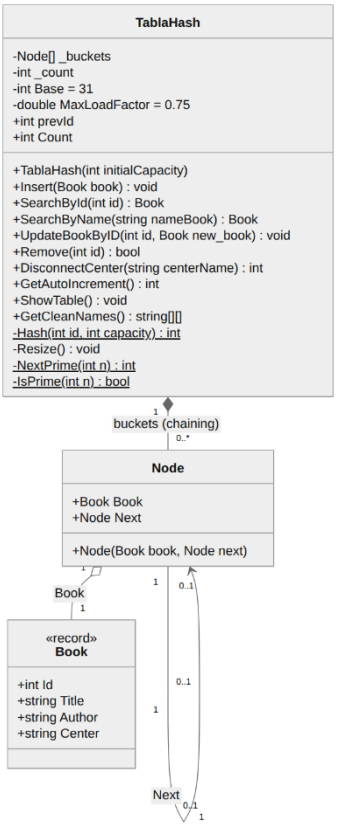
Rodrigo Andrés Pinto Ticú

1507325

Quetzaltenango, Quetzaltenango 29 de octubre del 2026

TABLA HASH

Diagrama de clase



En el sistema, se utilizó una tabla hash para el almacenamiento de libros dentro del sistema, esto se realizó con el fin de tener un acceso casi en tiempo constante.

La tabla hash consiste en un arreglo de nodos (a manera de lista enlazada) inicialmente con una capacidad fija (en el programa se tiene una capacidad inicial de 20 espacios). Cada que se desea insertar un elemento en la tabla se calcula el hash de su ID utilizando un algoritmo de hash polinomial (Pol(ID) % capacidad)

Obteniendo este hash se le asigna un espacio en la tabla, en caso el espacio esté ocupado se optó por utilizar el channing para concatenar los duplicados en la tabla, esto por el motivo que, a diferencia del sondeo lineal, con un buen algoritmo hash el channing puede ser casi imperceptible y tender a un big O de O(1).

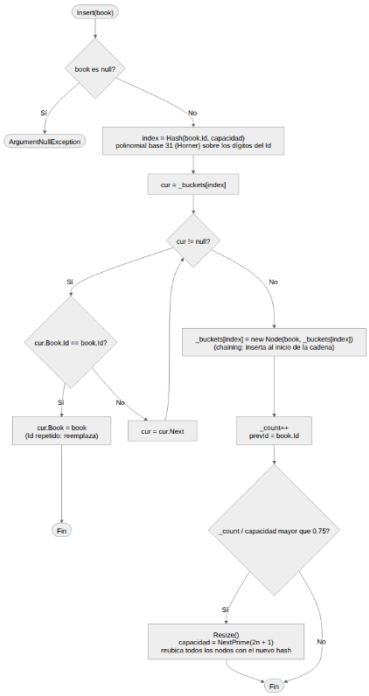
Al manejarse por arreglos, la tabla hash trabaja con un factor de carga del 75%, al llegar al factor de carga la tabla hash duplica su tamaño y realiza un re-hashing.

Comparación de operaciones con HashSet

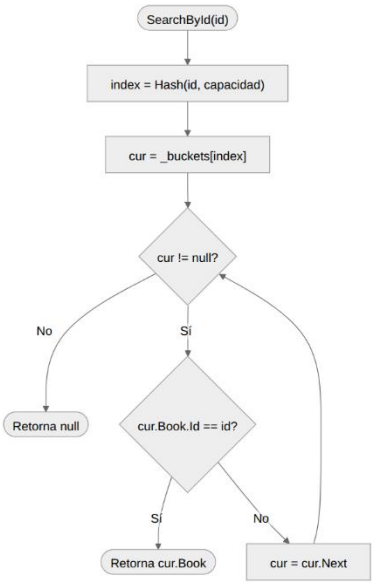
	HASHSET C#	Propia
Insert	Caso promedio O(1) Peor caso O(n)	Caso promedio O(1) Peor caso O(n)
Cointains /Search	Caso promedio O(1) Peor caso O(n)	Caso promedio O(1) Peor caso O(n)
Update	Caso promedio O(1) Peor caso O(n)	Caso promedio O(1) Peor caso O(n)
Remove	Caso promedio O(1) Peor caso O(n)	Caso promedio O(1) Peor caso O(n)

El caso promedio de todas las operaciones propias se debe a que la estructura en promedio y sin colisiones, retorna las coincidencias o los espacios vacíos al calcular el hash, pero cuando debe de hacer un resize o recorrer el channing, es que cualquier operación termina volviéndose O(n).

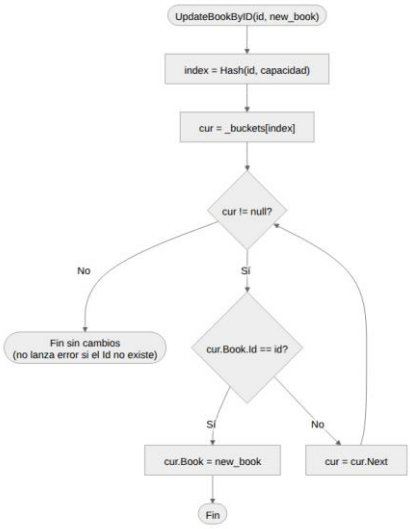
Diagramas:



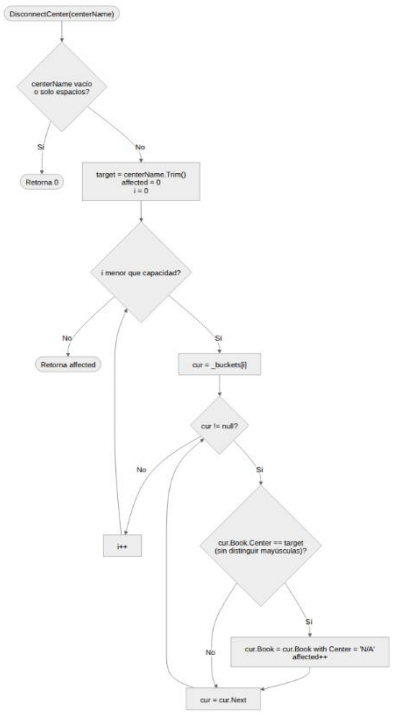
Create

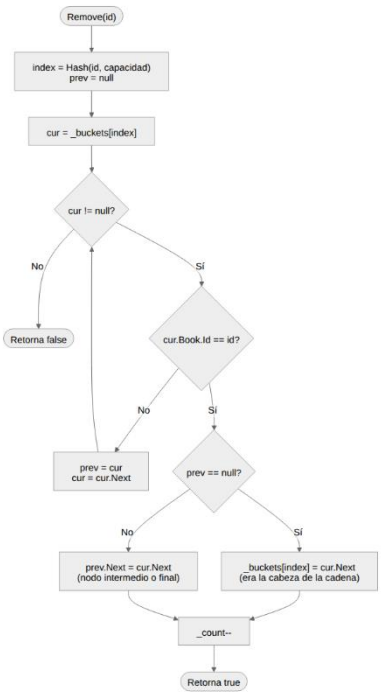


Search



Update

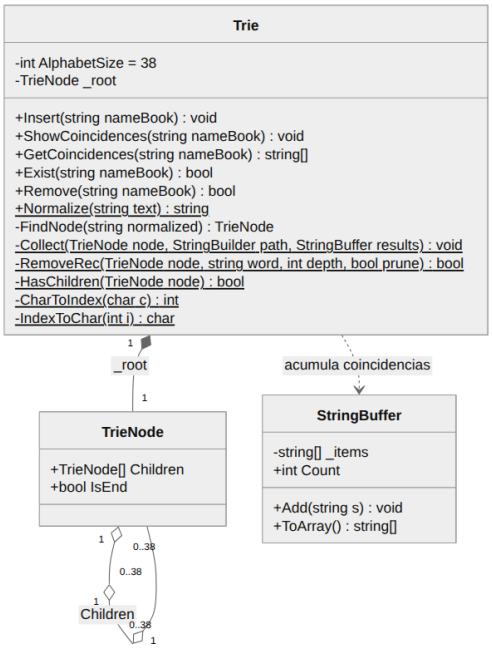




Delete

TRIES

Diagrama de clase



En el sistema, se utilizó dos tries para la búsqueda de nombres, de tal forma que la búsqueda no sea estrictamente exacta para localizar ya sea una sede o un libro.

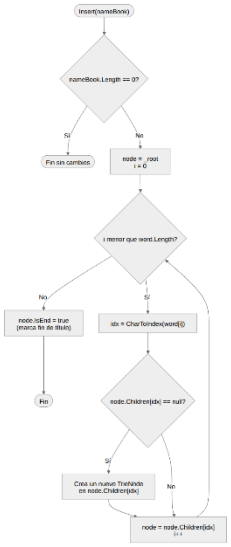
Al no utilizarse ninguna collection, se creó un Nodo (TrieNode) para actuar de igual manera como un árbol binario de búsqueda, con la diferencia que los hijos son un arreglo, además se creó una clase auxiliar StringBuffer para realizar la acumulación de las coincidencias cuando se lo necesite, acompañado de esto se creó un método para poder tener una conversión de caracteres a enteros, para facilitar el almacenamiento de las cadenas de texto (Nota: alphabetSize es un entero constante que pone limite a la cantidad de letras que puede soportar el tries, entre ellas se incluyó las letras minúsculas, números del 0-9, la ñ y el espacio, antes de insertar en el tries, todas las palabras son normalizadas reemplazando las vocales con tildes por sus respectivas, eliminando caracteres especiales y convirtiendo el texto en lower.

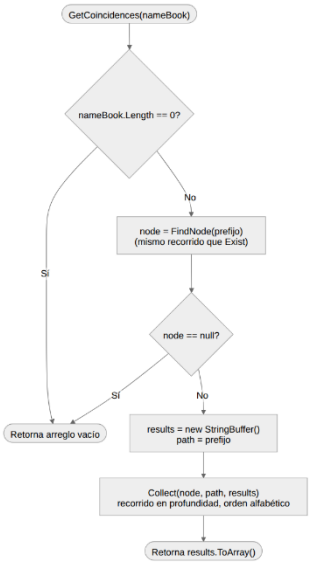
Comparación de operaciones con Tries utilizando Collections de .Net

	TRIES .NET	Propia
Insert	Caso promedio O(L) Caso especial O(L * ∑)	Caso promedio O(L) Caso especial O(L * ∑)
Cointains /Search	Caso promedio O(L) Caso especial O(1)	Caso promedio O(L) Caso especial O(1)
Update	Caso promedio O(L) Caso especial O(1) o O(L)	-
Remove	Caso promedio O(L) Caso especial O(1)	Caso promedio O(L) Caso especial O(L)

En la mayoría de casos, mi tries respeta la complejidad algorítmica basada en la longitud de la cadena almacenada, los casos especiales son provocados cuando se reserva una nueva rama en el tries (se tiene que reservar el alfabeto ∑ otra vez) o debe de recorrer únicamente el camino del prefijo, no implementé el update, debido a que en el sistema, no cuento con una opción especifica para actualizar los nombres registrados tanto de los libros como de las sedes.

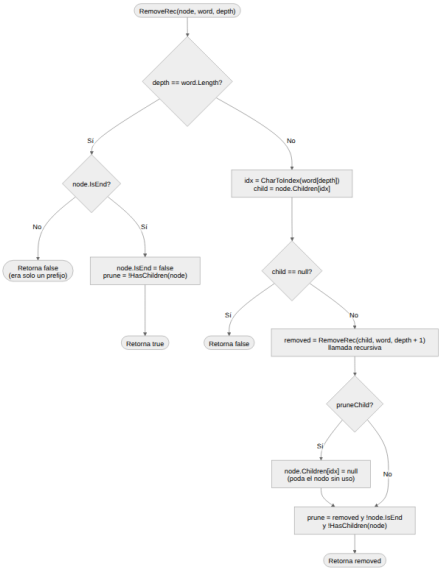
Diagramas:





Create

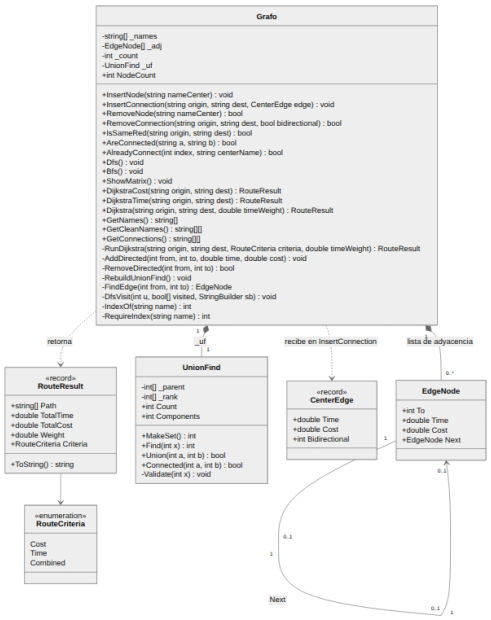
Read



Delete

GRAFO

Diagrama de clase



En el sistema utilizó el grafo para almacenar todas las sedes, de tal forma que se pueda calcular una ruta entre sedes a partir de las peticiones del usuario. Al igual que con las tablas hash, debido a la limitancia de no utilizar librerías como Collections, se utilizó arreglos para almacenar todos los nodos del grafo, se puede decir que con un factor de carga del 100% el espacio del arreglo se duplica para permitir agregar más sedes, para tener similitud entre el tries de búsqueda y el grafo, aquí también se normaliza las palabras removiendo cualquier símbolo fuera del alfabeto establecido, con esto se construyó paralelamente una estructura UnionFind, algo que sirve para poder saber si dos sedes pertenecen a la misma red.

Para agregar una conexión se utilizó de dos clases auxiliares, CenterEdge y EdgeNode. EdgeNode almacena a mandera de lista enlazada todas las conexiones que haya con un nodo, esto automáticamente genera la lista de adyacencia del grafo, lo que facilita su impresión al solo recorrer la misma de cada nodo y CenterEdge se utilizó a manera de almacenar los atributos de la arista del nodo donde se puede almacenar datos como el tiempo, costo y si la arista es bidireccional, la operación Find ayuda a detectar si una arista ya se encuentra conectando a dos nodos, lo que permite que no se pueda repetir la conexión.

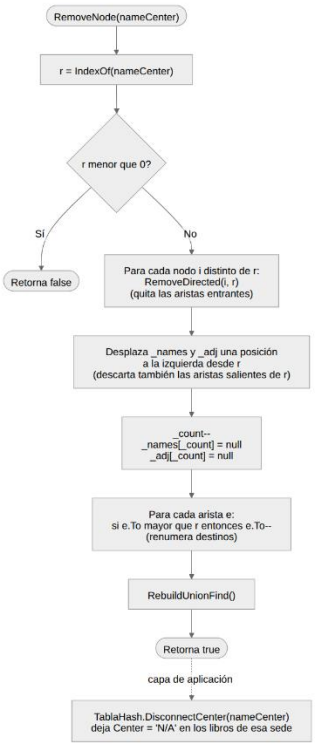
Comparación de operaciones con Grafo utilizando Collections de .Net

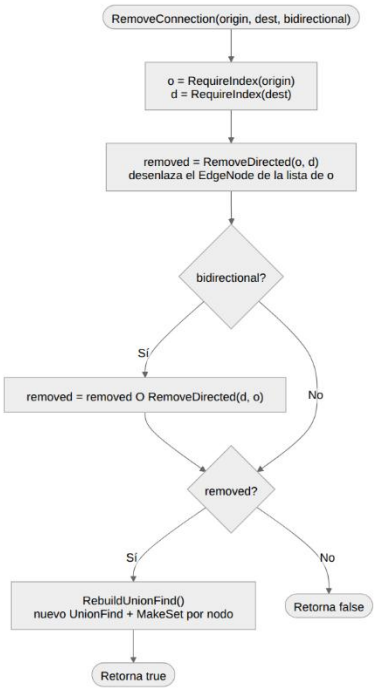
	GRAFO .NET	Propia
Insert Node	Caso promedio O(V)	Caso promedio O(V)
Insert Connection	Caso promedio O(V)	Caso promedio O(V)
Update	Caso promedio O(V)	Caso promedio O(V)
BFS	Caso promedio O(V)	Caso promedio O(V+E)
DFS	Caso promedio O(V)	Caso promedio O(V+E)
DISJKSTRA	Caso promedio O(V)	Caso promedio O(V^2)
Remove Node	Caso promedio O(V+E)	Caso promedio O(V+E)
Remove Connection	Caso promedio O(V)	Caso promedio O(V+E)

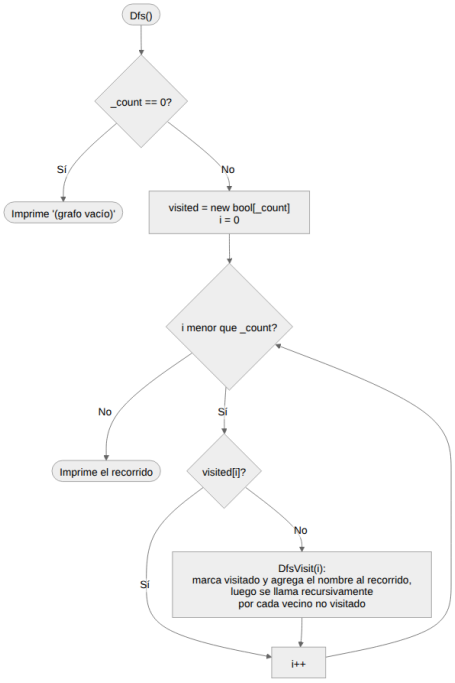
En este análisis donde V es el número de sedes (vertex) y E el número de aristas (edges), en la mayoría de las operaciones básicas se obtiene el mismo comportamiento debido a que los arreglos permite que en el caso necesario solo se recorra la cantidad de vértices o solo la cantidad de veritces

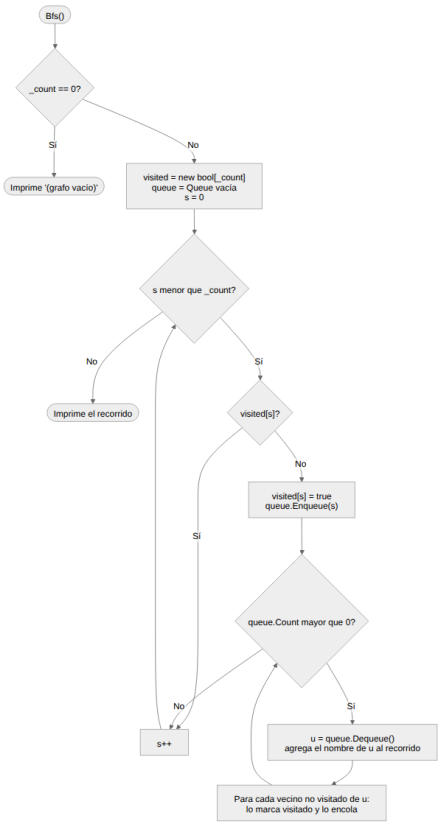
individual, no se construyó una operación para eliminar en el UnionFind, por lo mismo, al realizar una operación que necesite de su reconstrucción o de su construcción (como lo es con los recorridos y la eliminación) por lo que es necesario recorrer todas las aristas.

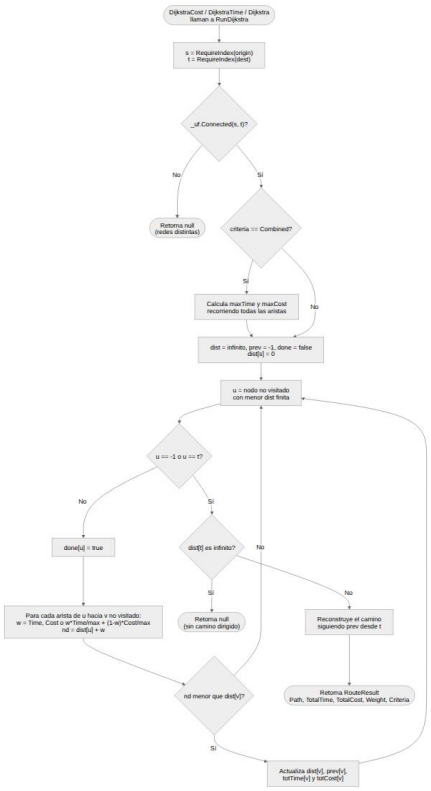
Diagramas:

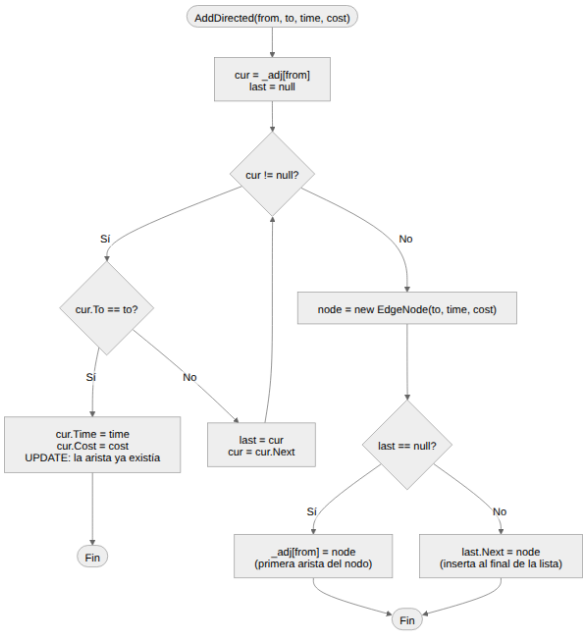

RemoveNode(nameCenter)
r = IndexOf(nameCenter)
r menor que 0?
Sí
No
Retorna false
Para cada nodo i distinto de r:
RemoveDirected(i, r)
(quita las aristas entrantes)
Desplaza _names y _adj una posición
a la izquierda desde r
(descarta también las aristas salientes de r)
_count--
_names[_count] = null
_adj[_count] = null
Para cada arista e:
si e.To mayor que r entonces e.To--
(renumera destinos)
RebuildUnionFind()
Retorna true
capa de aplicación
TablaHash.DisconnectCenter(nameCenter)
deja Center = 'N/A' en los libros de esa sede

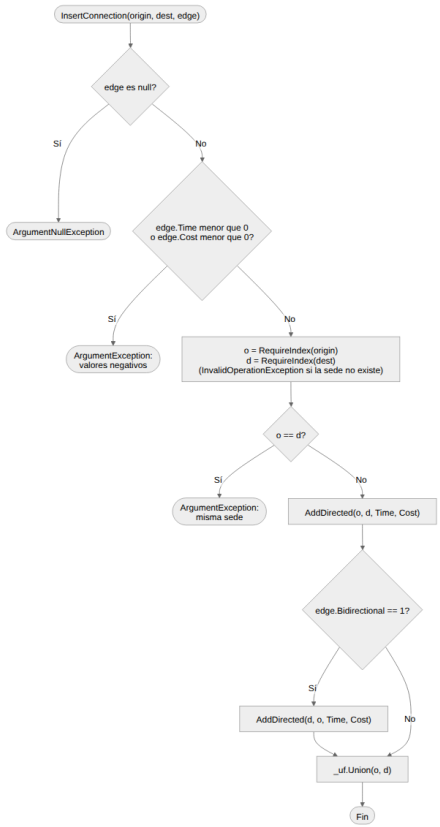

RemoveConnection(origin, dest, bidirectional)
o = RequireIndex(origin)
d = RequireIndex(dest)
removed = RemoveDirected(o, d)
desenlaza el EdgeNode de la lista de o
bidirectional?
Sí
No
removed = removed O RemoveDirected(d, o)
removed?
Sí
No
RebuildUnionFind()
nuevo UnionFind + MakeSet por nodo
Retorna false
Retorna true

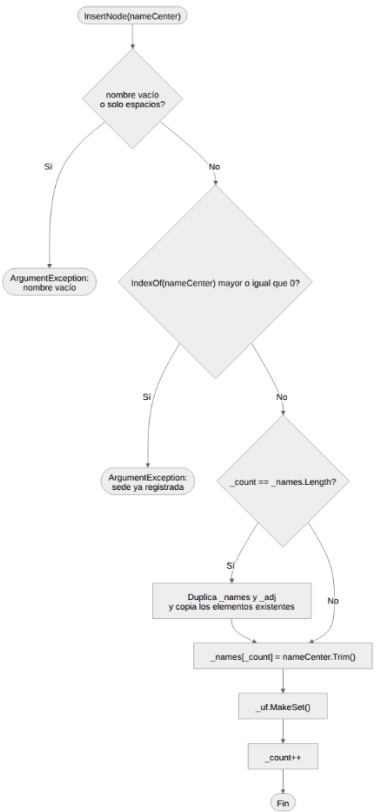

Dfs()
_count == 0?
Sí
No
Imprime '(grafo vacío)'
visited = new bool[_count]
i = 0
i menor que _count?
No
Sí
Imprime el recorrido
visited[i]?
No
Sí
DfsVisit(i):
marca visitado y agrega el nombre al recorrido,
luego se llama recursivamente
por cada vecino no visitado
i++


Bfs()
_count == 0?
Sí
No
Imprime '(grafo vacío)'
visited = new bool[_count]
queue = Queue vacía
s = 0
s menor que _count?
No
Sí
Imprime el recorrido
visited[s]?
No
Sí
visited[s] = true
queue.Enqueue(s)
queue.Count mayor que 0?
No
Sí
s++
u = queue.Dequeue()
agrega el nombre de u al recorrido
Para cada vecino no visitado de u:
lo marca visitado y lo encola


DijkstraCost / DijkstraTime / Dijkstra llaman a RunDijkstra
s = RequireIndex(origin)
t = RequireIndex(dest)
_uf.Connected(s, t)?
No
Sí
Retorna null (rutas distintas)
criteria == Combined?
Sí
No
Calcula maxTime y maxCost recorriendo todas las aristas
dist = infinito, prev = -1, done = false
dist[s] = 0
u = nodo no visitado con menor dist finita
u == -1 o u == t?
No
Sí
done[u] = true
dist[t] es infinito?
No
Sí
No
Para cada arista de u hacia v no visitado:
w = Time, Cost o w*Time/max + (1-w)*Cost/max
nd = dist[u] + w
Retorna null (sin camino dirigido)
Reconstruye el camino siguiendo prev desde t
nd menor que dist[v]?
Retorna RouteResult
Path, TotalTime, TotalCost, Weight, Criteria
Sí
Actualiza dist[v], prev[v], turTime[v] y turCost[v]


AddDirected(from, to, time, cost)
cur = _adj[from]
last = null
cur != null?
Sí
No
cur.To == to?
node = new EdgeNode(to, time, cost)
Sí
No
cur.Time = time
cur.Cost = cost
UPDATE: la arista ya existía
last = cur
cur = cur.Next
last == null?
Sí
No
Fin
_adj[from] = node
(primera arista del nodo)
last.Next = node
(inserta al final de la lista)
Fin


InsertConnection(origin, dest, edge)
edge es null?
Sí
No
ArgumentNullException
edge.Time menor que 0
o edge.Cost menor que 0?
Sí
No
ArgumentException:
valores negativos
o = RequireIndex(origin)
d = RequireIndex(dest)
(InvalidOperationException si la sede no existe)
o == d?
Sí
No
ArgumentException:
misma sede
AddDirected(o, d, Time, Cost)
edge.Bidirectional == 1?
Sí
No
AddDirected(d, o, Time, Cost)
_uf.Union(o, d)
Fin


InsertNode(nameCenter)
nombre vacío
o solo espacios?
Sí
No
ArgumentException:
nombre vacío
IndexOf(nameCenter) mayor o igual que 0?
Sí
No
ArgumentException:
sede ya registrada
_count == _names.Length?
Sí
No
Duplica _names y _adj
y copia los elementos existentes
_names[_count] = nameCenter.Trim()
_uf.MakeSet()
_count++
Fin

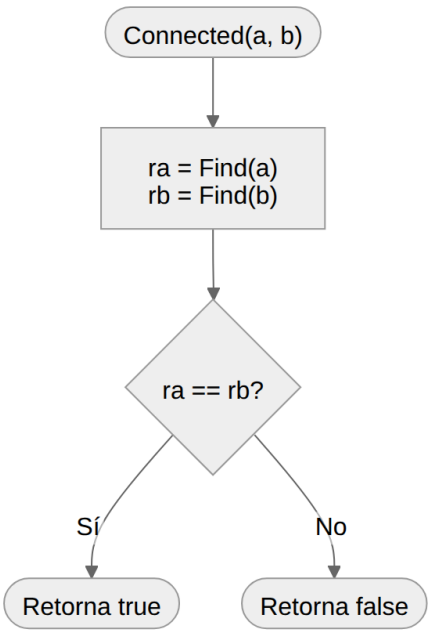
UNION FIND

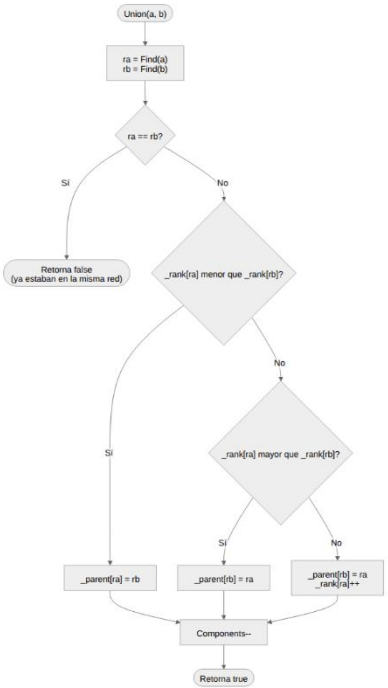
Diagrama de clase

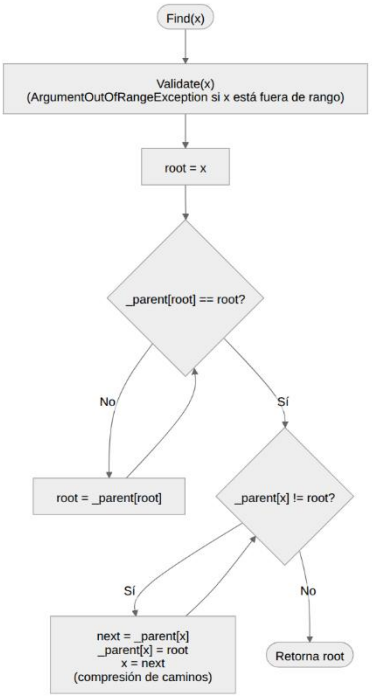
En todo el sistema UnionFind se utilizó más como una estructura auxiliar para los grafos y la ubicación de conexiones ya realizadas previamente o la verificación de que dos sedes se encuentren conectadas de alguna forma por lo mismo las operaciones fueron reducidas a: crear un conjunto con un set de datos, encontrar un dato en el conjunto y por consiguiente, encontrar dos datos en un conjunto (validar que dos sedes se encuentren en la misma red, implica que estén en el mismo conjunto Find(a) == Find(b)) y la operación Union, que comprueba si la operación anterior es valida para dos conjuntos A y B.

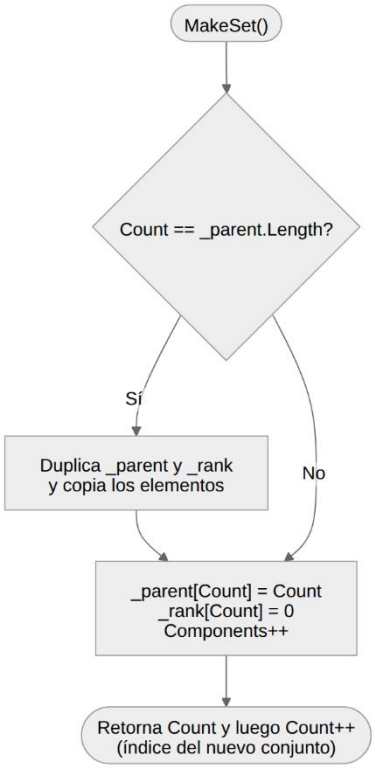
Para crear el conjunto se volvió a utilizar un arreglo que duplica su espacio cuando este se llena, en caso de que aún conserve espacios disponibles se agregan todos los datos (los ID autoincreméntales) al conjunto. Find secuencialmente recorre el conjunto buscando que exista alguna coincidencia, en caso de que la encuentre, retorna la coincidencia igual que en una lista enlazada. Union, internamente la operación find (Find(a) == Find(b) lo que implica que en el mismo conjunto se encuentran ambos registros, significa que sí están conectados), de tal forma que retorne true si es que aún no se encuentran los conjuntos unidos, o false si estos ya se encuentran unidos.

Diagramas:






Find(x)
Validate(x)
(ArgumentOutOfRangeException si x está fuera de rango)
root = x
_parent[root] == root?
No
Sí
root = _parent[root]
_parent[x] != root?
Sí
No
next = _parent[x]
_parent[x] = root
x = next
(compresión de caminos)
Retorna root


MakeSet()
Count == _parent.Length?
Sí
No
Duplica _parent y _rank
y copia los elementos
_parent[Count] = Count
_rank[Count] = 0
Components++
Retorna Count y luego Count++
(índice del nuevo conjunto)